

CASE STUDY / 00

Gamifying the break from routine

MiniQuest is a native Android application that translates response interruption and environmental enrichment into a pocket-sized RPG, using pervasive accelerometer sensing to reward the body for stepping outside its own habits.

AUTHOR

Shuhan Zhang

PLATFORM

Android · Kotlin · Jetpack Compose

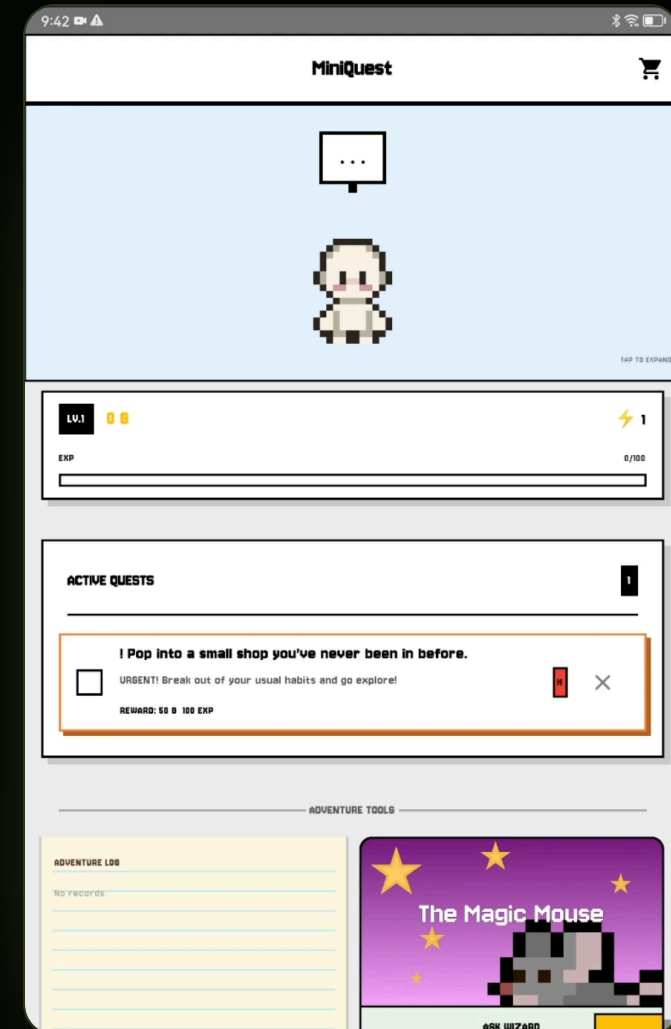
ROLE

Research · UX/UI · Android development

CONTEXT

University of York · Mobile Interaction

Complete case study. Research, interaction logic, product walkthrough and implementation are presented as one continuous system. Demo video and repository links appear on the closing page.



The rigidity problem

The opportunity was not another productivity tracker. It was a playful intervention for moments when repetition becomes a closed environment.

01 / PATTERN

Repetition is not confined to ASD

Stereotyped behavior describes repetitive action or sound with no apparent external purpose. Although strongly associated with autism, adults without ASD also fall into repetitive routines, at lower frequency and intensity.

Rapp & Vollmer, 2005 · Nwaordu & Charlton, 2024

02 / TRIGGER

Monotony reinforces the loop

Animal-behavior research links cage-induced stereotypies to monotonous, under-stimulating environments. Enriched and variable environments measurably reduce those patterns.

Gross et al., 2012 · Turner, Yang & Lewis, 2002

03 / DIRECTION

Interruption plus enrichment

Two behavioral strategies offered distinct product mechanics: redirect an active repetitive pattern, then broaden the environment with several low-friction activity choices.

Boyd, McDonough & Bodfish, 2012

A single-purpose habit tracker rewards you for staying inside the loop. MiniQuest is designed to reward you for stepping outside it.

Two routes beyond routine

The research became two complementary experience loops: one intervenes at the right moment; the other gives the user several inviting ways to change state.

01 / INTERRUPT

Urgent quests

When a stereotyped pattern would otherwise continue, the app interrupts it and redirects attention toward a new action. The Emergency Loop coroutine turns that moment into a short playable challenge.

01

Detect a closed loop

Routine continues without an external prompt.

02

Introduce novelty

An urgent quest arrives outside the planned list.

03

Reward redirection

A new action feeds back into XP and gold.

02 / ENRICH

Adventure tools

A menu of activity modules offers relaxation and exploration choices, adding the sensory and interactive variety monotonous days are missing.

MOVE

EXPLORE

ADVENTURE
TOOLS

REGULATE

ASK

Behavioral model → product system. Emergency quests handle response interruption; the Adventure Tools area handles environmental enrichment. Both earn XP or gold, so breaking routine remains legible inside one coherent RPG economy.

Beyond a gamified to-do list

Habitica proved that an RPG reward shell could motivate routine self-management. MiniQuest keeps the motivation, but changes what progress can mean.



Habitica: progress is primarily gated behind tasks the user has already written.

Habitica's reward loop stays locked to finishing pre-written tasks. Over time, that narrow focus can reproduce the same monotony the behavioral literature warns against: the user must already have planned their own escape from routine.

MiniQuest opens two additional channels. They allow progress to arrive through movement and system-induced novelty, without discarding the clarity of XP, gold and levels.

PASSIVE SENSING

Movement earns energy

Walking accrues progress in the background through raw accelerometer sensing, so the body can advance the game without another checklist.

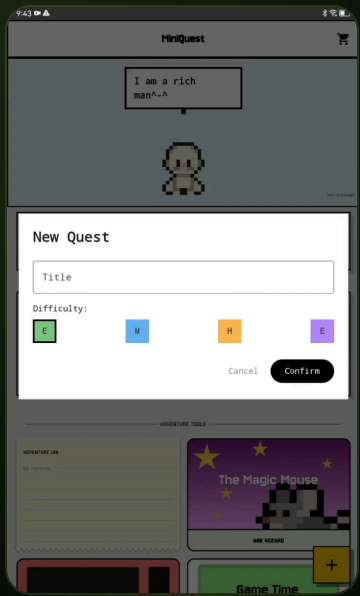
INDUCED NOVELTY

The system can surprise

MiniQuest, not the user's to-do list, decides when to surface an emergency task and redirect attention toward a new action.

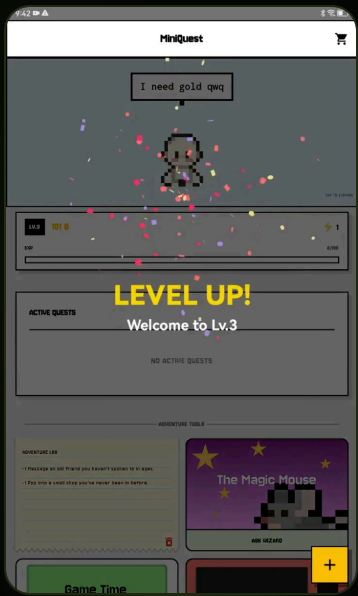
The core reward loop

Quest completion, progression feedback and low-friction task creation make the behavioral model understandable as an everyday game.



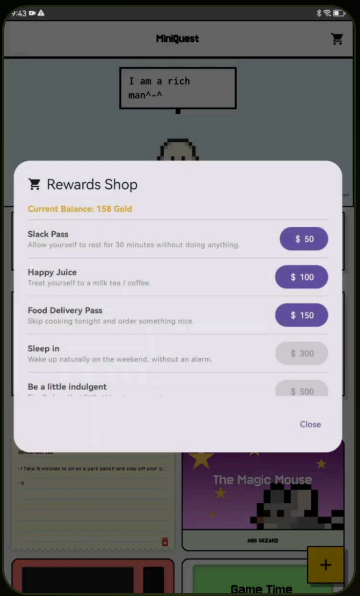
01 Define an action

A compact title-and-difficulty flow preserves the clarity of a useful task tool.



02 Earn progression

Confetti, XP and gold make completion feel like an event rather than a checkbox.

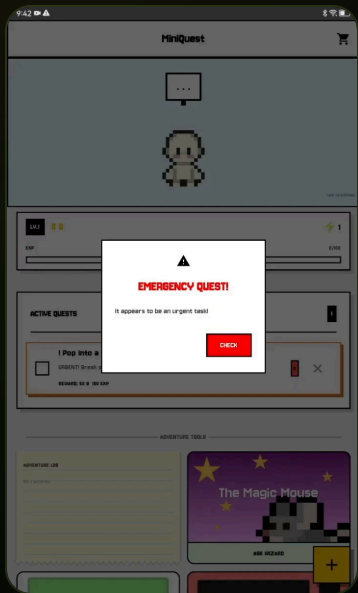


03 Spend the reward

The shop closes the loop by turning accumulated gold into visible outcomes.

An expanded adventure system

Beyond the core reward loop, three tools intervene, guide and regulate without asking the user to prepare another task in advance.



01 Emergency quest

Induced novelty interrupts the expected loop.



02 Wizard's Hall

A role-play wrapper for asynchronous guidance.



03 Stress relief

A one-tap, haptic route to sensory regulation.

Friendly enough to interrupt

Three reference points shaped how MiniQuest looks, rewards and redirects attention without adopting a clinical tone.



01 / MOTIVATION

Habitica's RPG reward loop

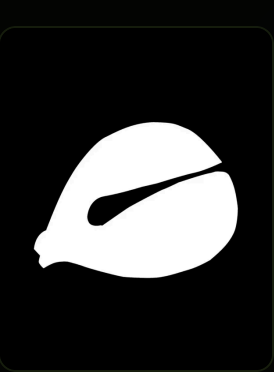
Habitica demonstrated that XP and gold can make routine self-management motivating. MiniQuest retains that clarity while allowing movement and system-generated novelty to produce progress.



02 / CHARACTER

Hard-edged pixel aesthetic

Classic sprite-based RPGs feel playful and low-stakes. Vector pixel fonts and a custom HardShadowBox resist high-DPI smoothing, keeping the game's edge crisp rather than approximate.



03 / REGULATION

One-tap ritual

The wooden fish is a repetitive meditative object. Its low-demand interaction informed a stress-relief module that pairs a single tap with haptic confirmation instead of asking for more visual attention.

Offline-first, reactive architecture

MVVM and unidirectional data flow keep persistence, sensors, network intelligence and Compose UI synchronized around one source of truth.

VIEW

Jetpack Compose UI

A passive consumer maps StateFlow to the screen and forwards user intent upward. No business logic lives in the view.

VIEWMODEL

AppViewModel

Holds immutable StateFlow, orchestrates repository calls, sensor input and AI requests, and scopes coroutines to viewModelScope.

MODEL

Focused data services

QuestRepository, StepSensorManager and OkClient each hold one responsibility and remain unaware of the UI.

Why Room + Flow. Data is written locally first for instant offline play. Repository changes stream into Compose automatically, so an inserted emergency quest redraws the interface without a manual refresh.

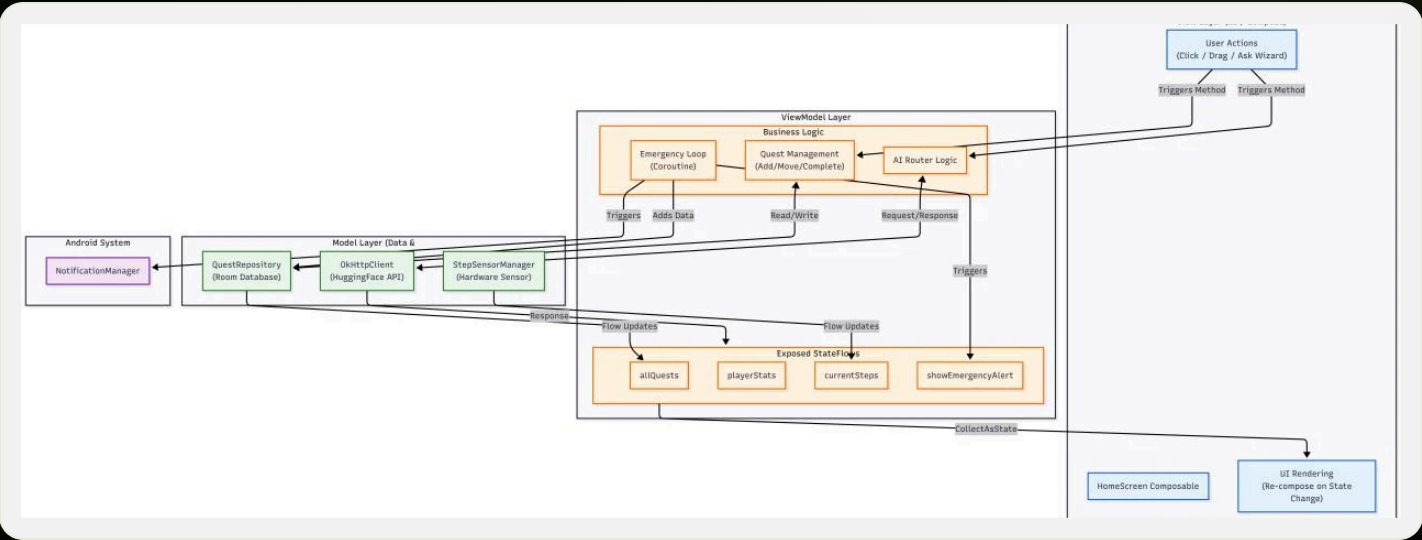


Fig. A1 · Original architecture diagram submitted with the technical report.

Pervasive sensing from raw acceleration

A compatibility-led sensing layer avoids dependence on the dedicated step-counter hardware missing from many lower-end and older Android devices.

MiniQuest reads the raw tri-axis accelerometer, present on nearly every Android phone, and computes motion magnitude itself. Step detection is therefore not limited by device tier.

Orientation-independent signal. The vector magnitude $\sqrt{(x^2 + y^2 + z^2)}$ turns motion into one scalar value, however the phone is carried.

Noise filtering. A 300ms window stops a single shake from being counted as several steps in a row.

StepSensorManager.kt · magnitude

```
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        val x = it.values[0]
        val y = it.values[1]
        val z = it.values[2]
        val magnitude = sqrt(x * x + y * y + z * z)
    }
}
```

StepSensorManager.kt · debounce

```
val currentTime = System.currentTimeMillis()
if (currentTime - lastShakeTime > 300) {
    internalSteps++
    saveStepsToStorage()
    lastShakeTime = currentTime
}
```

Multimodal and sensory regulation

MiniQuest treats tactility and visual character as functional channels: one reduces visual demand; the other makes interruption feel playful rather than punitive.



Async intelligence that fails gracefully

Because network latency is unpredictable, the request runs in Kotlin Coroutines rather than AsyncTask or a heavier RxJava dependency. viewModelScope automatically cancels a suspended request when the user leaves the screen, avoiding leaks and wasted battery.

Idle

The dialogue is available and waiting on the user.

Thinking

The request runs on Dispatchers.IO; the UI locks and shows "...".

Reply

The API result resolves, or the offline grimoire supplies a fallback.

Uncertainty control. The thinking flag prevents duplicate requests while the user is waiting for an answer.

The Grand Wizard wraps a live HuggingFace request in a character-driven interaction while lifecycle-safe coroutines and an offline grimoire protect continuity.

```
fun askGrandWizard(userQuestion: String) {
    viewModelScope.launch {
        // enter "thinking" and lock the UI
        _isWizardThinking.value = true
        _wizardReply.value = "..."

        var finalResult: String? = null
        for (modelId in routerModels) {
            val (success, result) =
                tryRouterModel(modelId, userQuestion)
            if (success && !result.isNullOrEmpty()) {
                finalResult = result
                break
            }
        }

        // resolve with an offline fallback
        _wizardReply.value =
            finalResult ?: offlineGrimoire.random()
        _isWizardThinking.value = false
    }
}
```

AppViewModel.kt · askGrandWizard()

Stack, constraints and next steps

The submitted prototype is a working Android system with explicit permissions, local persistence and clear boundaries for future validation.

Permission	Usage
Internet	OkHttp calls to the HuggingFace API powering Wizard Q&A.
Network State	Checks Wi-Fi or mobile connectivity before an AI request.
Vibrate	Haptic feedback for the stress-relief module.
Post Notifications	System pop-ups for emergency quest alerts.
Body Sensor	Reserved for biometric data in preparation for future expansion.

LIBRARIES

Small, focused dependencies

OkHttp handles data transmission; Konfetti turns quest completion into a visible reward event.

CURRENT LIMITATIONS

Local data and sensing ceiling

- Uninstalling the app deletes Room-stored progress.
- Phone-only acceleration cannot use wearables and offers weak resistance to mechanical step-cheating.

PLANNED NEXT

Continuity and stronger signals

- Firebase account login and multi-device synchronization.
- Health Connect wearable data plus ML Kit waveform analysis to filter mechanical cheating.

Engineering and experience, held together

The final value is not one feature in isolation, but the balance between a credible mobile system and a deliberately playful intervention.

MiniQuest is not simply a task manager wearing an RPG skin. It is a working test of whether phone sensors can be pointed at a behavioral-health problem instead of another to-do list.

By pairing a self-built perception layer with an offline-first MVVM foundation and lifecycle-safe coroutine concurrency, the project makes its immersive pixel-art feel rest on a coherent technical system.

INDIVIDUAL CONTRIBUTION

Behavioral research, concept development, interaction design, visual direction, Kotlin implementation, testing and final case-study curation.



WATCH

Test video

youtube.com/shorts/CBCdgtMNU3ZE



EXPLORE

Project GitHub

github.com/Shuhan0001/MiniQuest